

```
In [1]: #Step 1: Import Libraries
```

```
In [3]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.stats import ttest_ind

from sklearn.preprocessing import LabelEncoder
```

```
In [4]: #Step 2: Load Dataset
```

```
In [5]: df = pd.read_csv("ecommerce_powerbi_data.csv")

print(df.head())
```

	ID	Customer_ID	Order_Date	Order_Month	Order_Quarter	Warehouse_Block	
\							
0	1	4174	2024-11-23	11	4		D
1	2	4507	2023-01-17	1	1		F
2	3	1860	2024-01-03	1	1		A
3	4	2294	2023-05-05	5	2		B
4	5	2130	2023-11-08	11	4		C
		Shipment_Mode	Product_Category	Product_Importance	Gender	...	\
0		Flight	Fashion	low	F	...	
1		Flight	Electronics	low	M	...	
2		Flight	Beauty	low	M	...	
3		Flight	Electronics	medium	M	...	
4		Flight	Sports	medium	F	...	
		Prior_Purchases	Delivery_Status	Delivery_Label	Delivery_Days		\
0		3	1	On-Time	4		
1		2	1	On-Time	5		
2		4	1	On-Time	4		
3		4	1	On-Time	6		
4		3	1	On-Time	6		
		Risk_Score	Risk_Score_Value	Return_Risk	Return_Flag		\
0		medium	2	Medium	Yes		
1		low	1	Low	No		
2		medium	2	Medium	Yes		
3		low	1	Low	No		
4		medium	2	Medium	Yes		
		Estimated_Revenue_Loss	Customer_Segment				
0		26.55	Discount-Driven				
1		0.00	Discount-Driven				
2		27.45	Discount-Driven				
3		0.00	Loyal Customer				
4		27.60	Discount-Driven				

[5 rows x 25 columns]

```
In [6]: #Step 3: Dataset Information
```

```
In [7]: print(df.shape)
```

```
(10999, 25)
```

```
In [8]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10999 entries, 0 to 10998
Data columns (total 25 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   ID                                    10999 non-null  int64
 1   Customer_ID                          10999 non-null  int64
 2   Order_Date                           10999 non-null  object
 3   Order_Month                          10999 non-null  int64
 4   Order_Quarter                        10999 non-null  int64
 5   Warehouse_Block                      10999 non-null  object
 6   Shipment_Mode                        10999 non-null  object
 7   Product_Category                     10999 non-null  object
 8   Product_Importance                   10999 non-null  object
 9   Gender                               10999 non-null  object
10   Weight_gms                           10999 non-null  int64
11   Product_Cost                         10999 non-null  int64
12   Discount                             10999 non-null  int64
13   Customer_Rating                      10999 non-null  int64
14   Customer_Care_Calls                  10999 non-null  int64
15   Prior_Purchases                      10999 non-null  int64
16   Delivery_Status                      10999 non-null  int64
17   Delivery_Label                       10999 non-null  object
18   Delivery_Days                        10999 non-null  int64
19   Risk_Score                           10999 non-null  object
20   Risk_Score_Value                     10999 non-null  int64
21   Return_Risk                          10999 non-null  object
22   Return_Flag                          10999 non-null  object
23   Estimated_Revenue_Loss                10999 non-null  float64
24   Customer_Segment                     10999 non-null  object
dtypes: float64(1), int64(13), object(11)
memory usage: 2.1+ MB
None
```

```
In [9]: print(df.describe())
```

	ID	Customer_ID	Order_Month	Order_Quarter	Weight_gm
s \ count	10999.000000	10999.000000	10999.000000	10999.000000	10999.000000
0					
mean	5500.000000	3010.722702	6.459405	2.490135	3634.01672
9					
std	3175.28214	1153.966424	3.427869	1.109510	1635.37725
1					
min	1.000000	1000.000000	1.000000	1.000000	1001.000000
0					
25%	2750.500000	2001.500000	3.500000	1.500000	1839.500000
0					
50%	5500.000000	3007.000000	6.000000	2.000000	4149.000000
0					
75%	8249.500000	4014.000000	9.000000	3.000000	5050.000000
0					
max	10999.000000	4999.000000	12.000000	4.000000	7846.000000
0					

	Product_Cost	Discount	Customer_Rating	Customer_Care_Calls \
count	10999.000000	10999.000000	10999.000000	10999.000000
mean	210.196836	13.373216	2.990545	4.054459
std	48.063272	16.205527	1.413603	1.141490
min	96.000000	1.000000	1.000000	2.000000
25%	169.000000	4.000000	2.000000	3.000000
50%	214.000000	7.000000	3.000000	4.000000
75%	251.000000	10.000000	4.000000	5.000000
max	310.000000	65.000000	5.000000	7.000000

	Prior_Purchases	Delivery_Status	Delivery_Days	Risk_Score_Value
\ count	10999.000000	10999.000000	10999.000000	10999.000000
mean	3.567597	0.596691	8.680971	1.907082
std	1.522860	0.490584	5.189687	0.700650
min	2.000000	0.000000	3.000000	1.000000
25%	3.000000	0.000000	5.000000	1.000000
50%	3.000000	1.000000	7.000000	2.000000
75%	4.000000	1.000000	13.000000	2.000000
max	10.000000	1.000000	20.000000	3.000000

	Estimated_Revenue_Loss
count	10999.000000
mean	22.349441
std	15.713486
min	0.000000
25%	0.000000
50%	26.700000
75%	35.850000
max	46.500000

```
In [10]: #Step 4: Missing Values
```

```
In [11]: print(df.isnull().sum())
```

ID	0
Customer_ID	0
Order_Date	0
Order_Month	0
Order_Quarter	0
Warehouse_Block	0
Shipment_Mode	0
Product_Category	0
Product_Importance	0
Gender	0
Weight_gms	0
Product_Cost	0
Discount	0
Customer_Rating	0
Customer_Care_Calls	0
Prior_Purchases	0
Delivery_Status	0
Delivery_Label	0
Delivery_Days	0
Risk_Score	0
Risk_Score_Value	0
Return_Risk	0
Return_Flag	0
Estimated_Revenue_Loss	0
Customer_Segment	0
dtype:	int64

```
In [12]: #Step 5: Duplicate Records
```

```
In [13]: print(df.duplicated().sum())

df.drop_duplicates(inplace=True)

0
```

```
In [14]: #Phase 1: Data Understanding
```

```
In [15]: total_orders = len(df)

print("Total Orders:", total_orders)

Total Orders: 10999
```

```
In [16]: returned = df[df['Return_Flag']=="Yes"]

print("Returned Orders:", len(returned))

Returned Orders: 7741
```

```
In [17]: return_rate = (len(returned)/len(df))*100

print("Return Rate:", round(return_rate,2),"%")

Return Rate: 70.38 %
```

```
In [33]: #Phase 2: Statistical Analysis
```

```
In [18]: print(df.mean(numeric_only=True))
```

ID	5500.000000
Customer_ID	3010.722702
Order_Month	6.459405
Order_Quarter	2.490135
Weight_gms	3634.016729
Product_Cost	210.196836
Discount	13.373216
Customer_Rating	2.990545
Customer_Care_Calls	4.054459
Prior_Purchases	3.567597
Delivery_Status	0.596691
Delivery_Days	8.680971
Risk_Score_Value	1.907082
Estimated_Revenue_Loss	22.349441

dtype: float64

```
In [19]: print(df.median(numeric_only=True))
```

ID	5500.0
Customer_ID	3007.0
Order_Month	6.0
Order_Quarter	2.0
Weight_gms	4149.0
Product_Cost	214.0
Discount	7.0
Customer_Rating	3.0
Customer_Care_Calls	4.0
Prior_Purchases	3.0
Delivery_Status	1.0
Delivery_Days	7.0
Risk_Score_Value	2.0
Estimated_Revenue_Loss	26.7

dtype: float64

```
In [20]: print(df.std(numeric_only=True))
```

ID	3175.282140
Customer_ID	1153.966424
Order_Month	3.427869
Order_Quarter	1.109510
Weight_gms	1635.377251
Product_Cost	48.063272
Discount	16.205527
Customer_Rating	1.413603
Customer_Care_Calls	1.141490
Prior_Purchases	1.522860
Delivery_Status	0.490584
Delivery_Days	5.189687
Risk_Score_Value	0.700650
Estimated_Revenue_Loss	15.713486

dtype: float64

```
In [21]: print(df.quantile([0.25,0.5,0.75],numeric_only=True))
```

	ID	Customer_ID	Order_Month	Order_Quarter	Weight_gms	\
0.25	2750.5	2001.5	3.5	1.5	1839.5	
0.50	5500.0	3007.0	6.0	2.0	4149.0	
0.75	8249.5	4014.0	9.0	3.0	5050.0	

	Product_Cost	Discount	Customer_Rating	Customer_Care_Calls	\
0.25	169.0	4.0	2.0	3.0	
0.50	214.0	7.0	3.0	4.0	
0.75	251.0	10.0	4.0	5.0	

	Prior_Purchases	Delivery_Status	Delivery_Days	Risk_Score_Value	\
0.25	3.0	0.0	5.0	1.0	
0.50	3.0	1.0	7.0	2.0	
0.75	4.0	1.0	13.0	2.0	

	Estimated_Revenue_Loss
0.25	0.00
0.50	26.70
0.75	35.85

```
In [22]: print(df['Delivery_Days'].mean())
```

8.680970997363398

```
In [23]: print(df['Product_Cost'].mean())
```

210.19683607600692

```
In [24]: print(df['Customer_Rating'].mean())
```

2.9905445949631786

```
In [32]: #Phase 3: Return Pattern Investigation
```

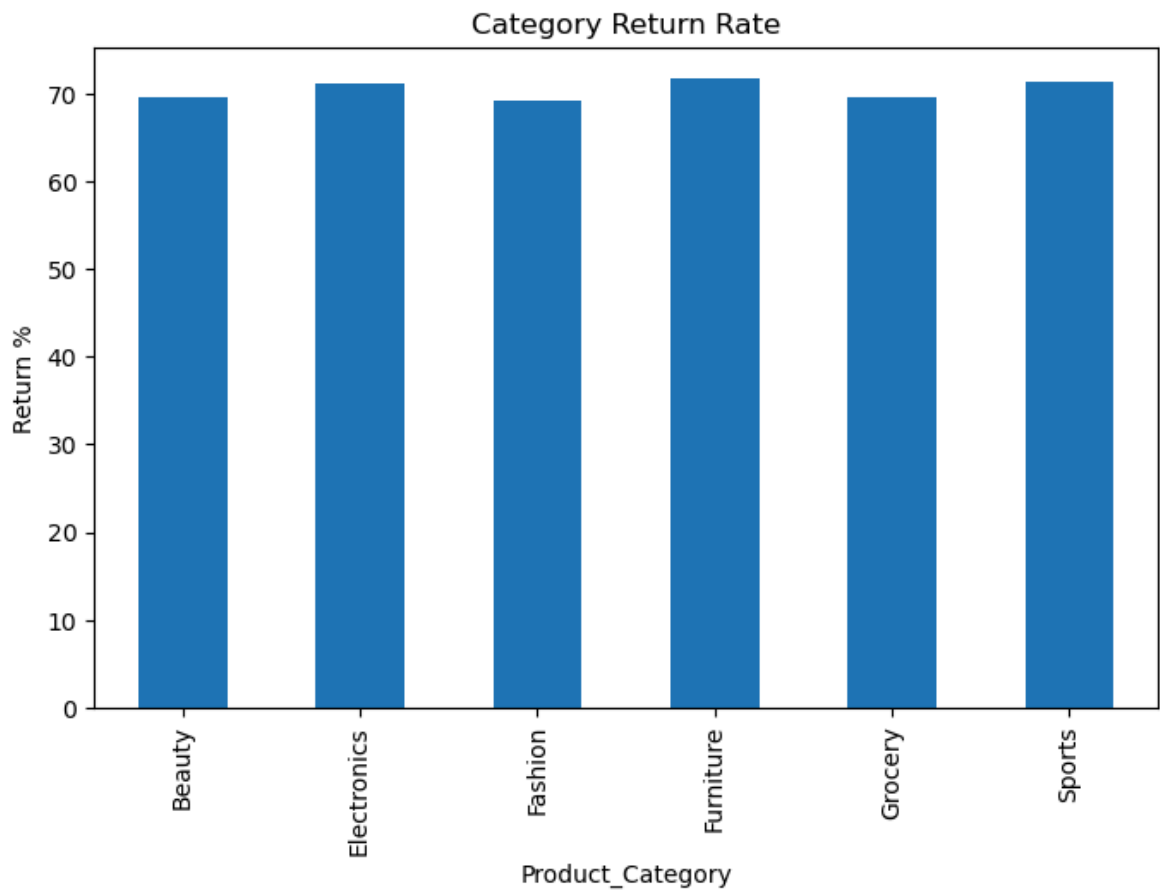
```
In [25]: category_return = pd.crosstab(
    df['Product_Category'],
    df['Return_Flag'],
    normalize='index'
)*100

print(category_return)

category_return['Yes'].plot(
    kind='bar',
    figsize=(8,5)
)

plt.title("Category Return Rate")
plt.ylabel("Return %")
plt.show()
```

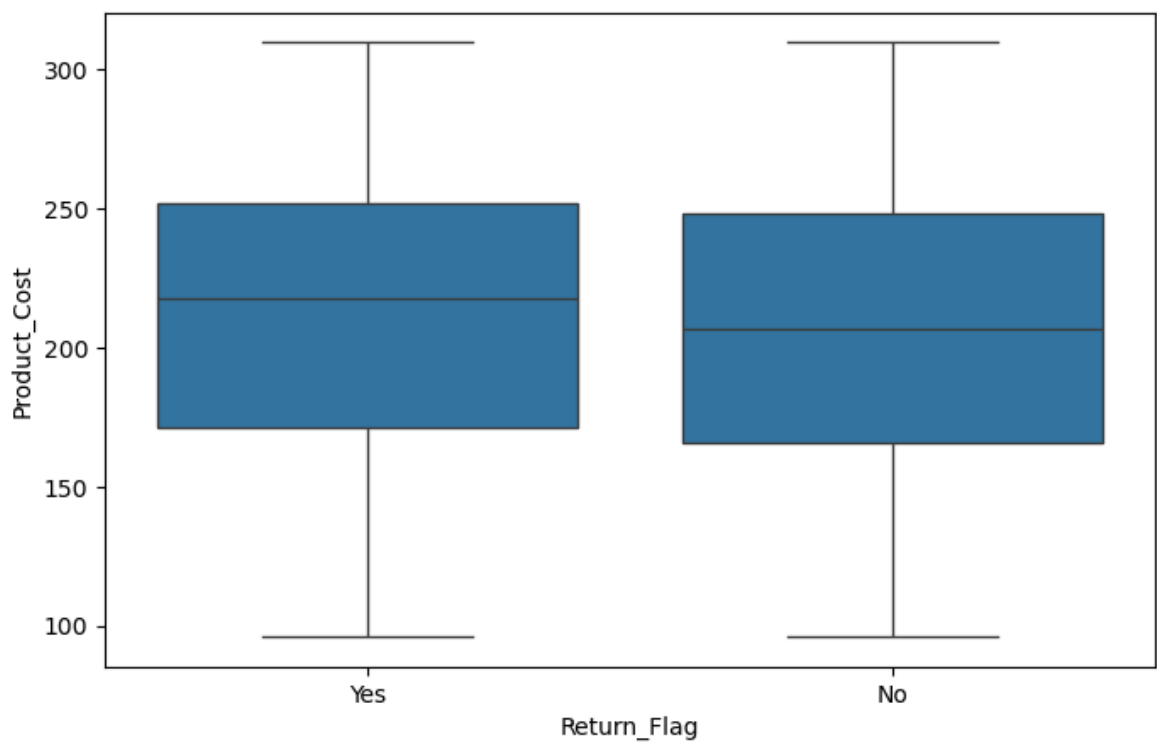
Return_Flag	No	Yes
Product_Category		
Beauty	30.509383	69.490617
Electronics	28.855454	71.144546
Fashion	30.748663	69.251337
Furniture	28.361459	71.638541
Grocery	30.511364	69.488636
Sports	28.737542	71.262458



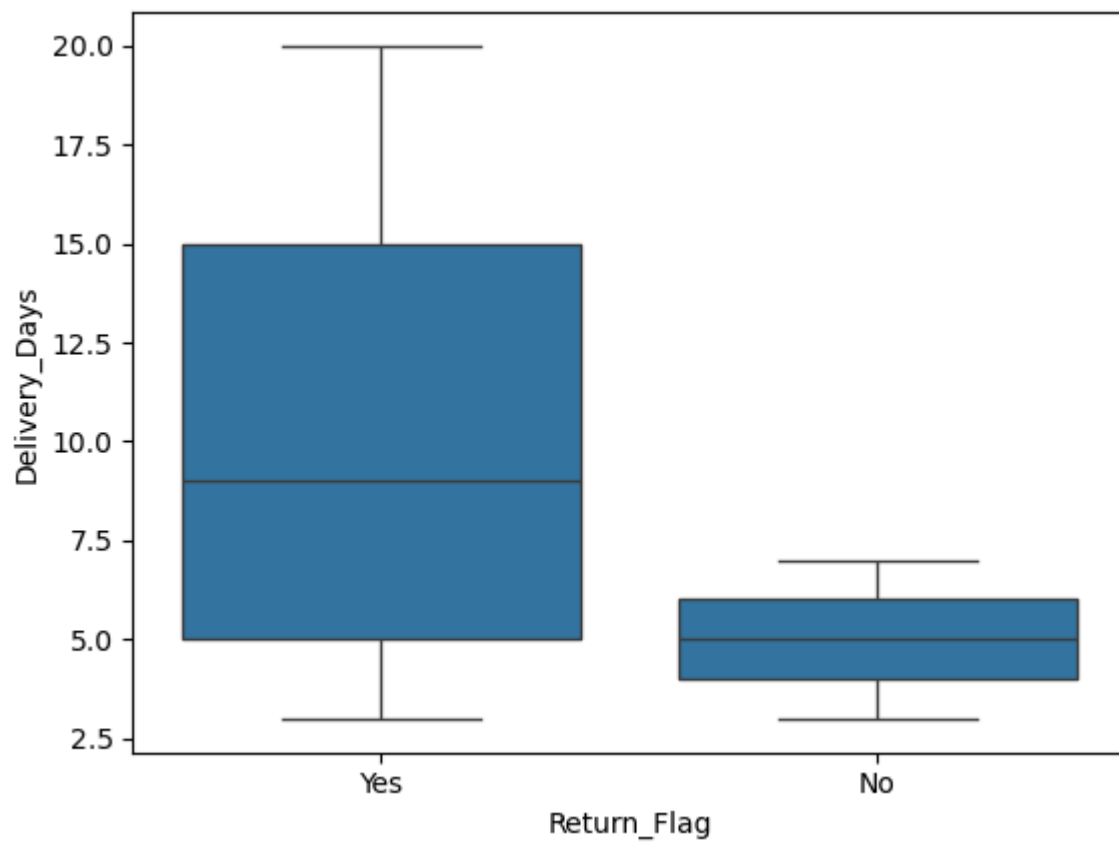
```
In [26]: plt.figure(figsize=(8,5))
```

```
sns.boxplot(  
    x='Return_Flag',  
    y='Product_Cost',  
    data=df  
)
```

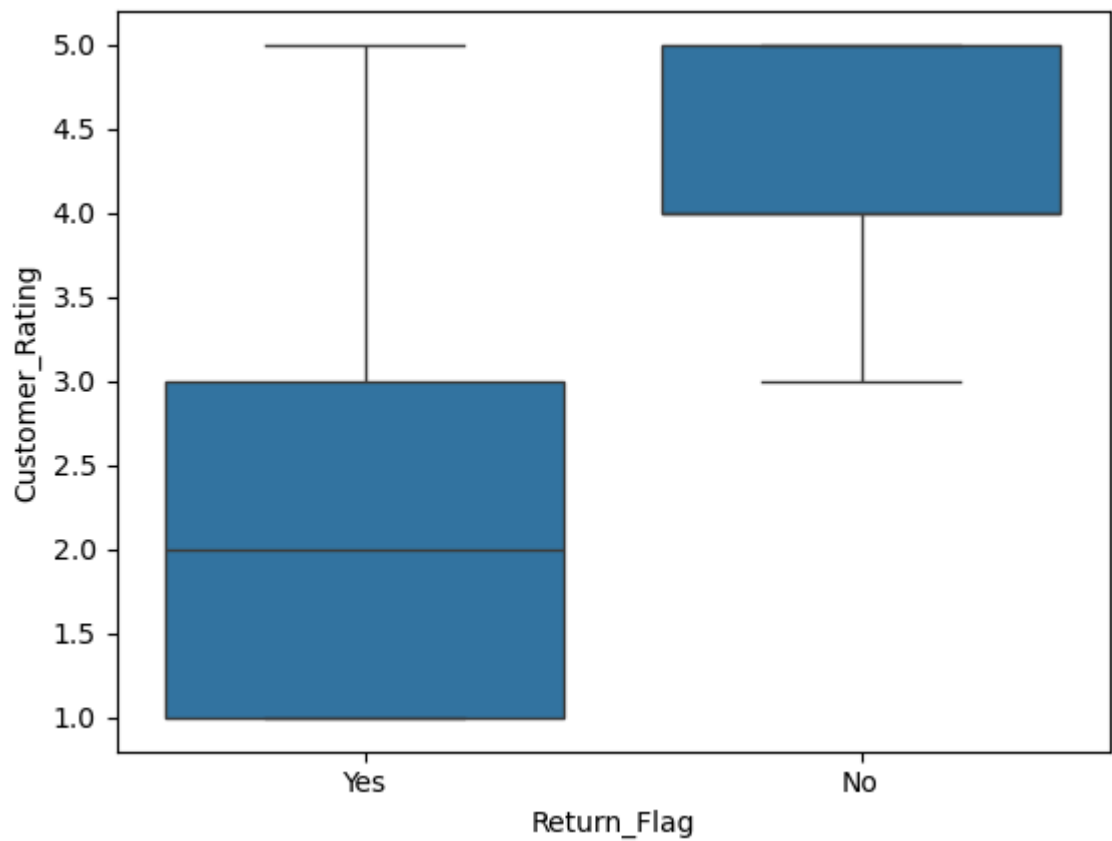
```
plt.show()
```



```
In [27]: sns.boxplot(  
    x='Return_Flag',  
    y='Delivery_Days',  
    data=df  
)  
  
plt.show()
```



```
In [28]: sns.boxplot(  
    x='Return_Flag',  
    y='Customer_Rating',  
    data=df  
)  
  
plt.show()
```

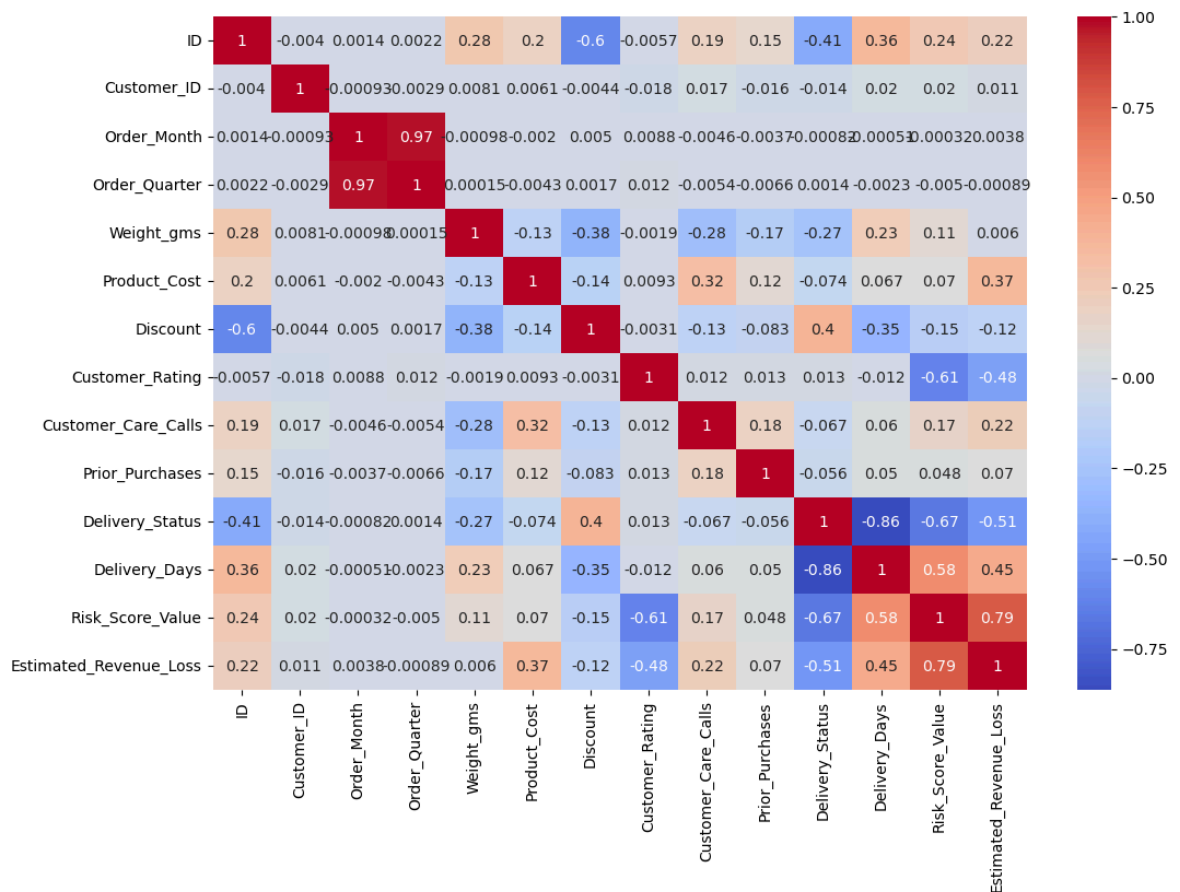


```
In [29]: numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(12,8))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.show()
```



```
In [30]: #Phase 4: Outlier Detection
```

```
In [31]: Q1 = df['Delivery_Days'].quantile(0.25)
Q3 = df['Delivery_Days'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers = df[
    (df['Delivery_Days'] < lower) |
    (df['Delivery_Days'] > upper)
]

print(outliers.shape)
```

(0, 25)

```
In [34]: z = np.abs(stats.zscore(df['Product_Cost']))

outliers = df[z>3]

print(outliers.shape)
```

(0, 25)

```
In [35]: #Phase 5: Hypothesis Testing
```

```
In [36]: returned = df[df['Return_Flag']=="Yes"]['Delivery_Days']

not_returned = df[df['Return_Flag']=="No"]['Delivery_Days']
```

```
t,p = ttest_ind(
    returned,
    not_returned
)

print("T Statistic:",t)
print("P Value:",p)
```

T Statistic: 54.41067212477698
P Value: 0.0

```
In [37]: if p < 0.05:
        print("Reject H0")
        else:
        print("Fail to Reject H0")
```

Reject H0

```
In [38]: #Phase 6: Customer Segmentation
```

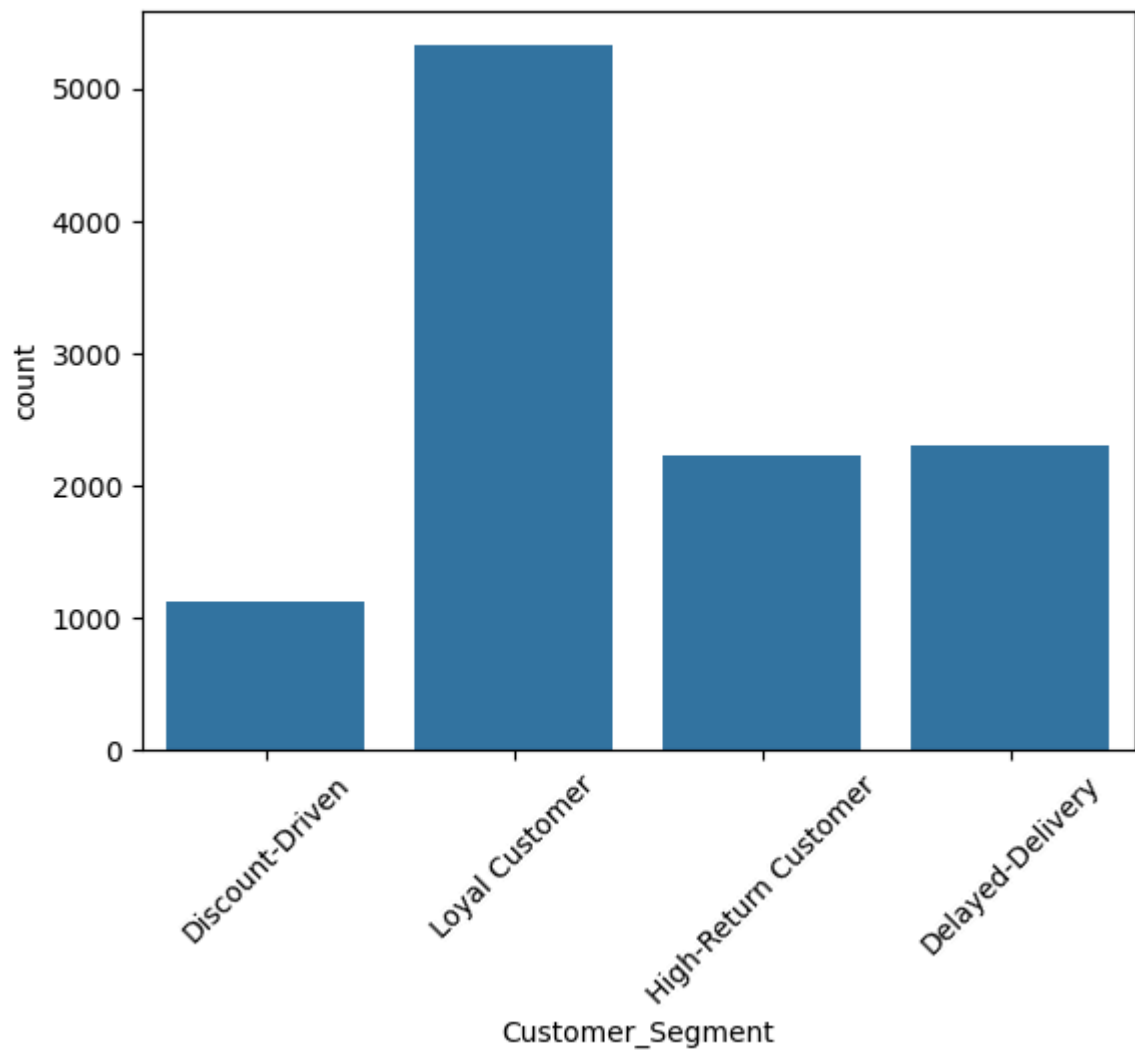
```
In [39]: segment = df.groupby(
        'Customer_Segment'
    ).agg({
        'Return_Flag':'count',
        'Estimated_Revenue_Loss':'sum'
    })

    print(segment)
```

	Return_Flag	Estimated_Revenue_Loss
Customer_Segment		
Delayed-Delivery	2309	73331.70
Discount-Driven	1131	22359.75
High-Return Customer	2236	72630.75
Loyal Customer	5323	77499.30

```
In [40]: sns.countplot(
        data=df,
        x='Customer_Segment'
    )

    plt.xticks(rotation=45)
    plt.show()
```



In []: *#Phase 7: Business Insights*

```
In [41]: print("Top Return Categories")

print(
category_return.sort_values(
'Yes',
ascending=False
))
```

Top Return Categories

Return_Flag	No	Yes
Product_Category		
Furniture	28.361459	71.638541
Sports	28.737542	71.262458
Electronics	28.855454	71.144546
Beauty	30.509383	69.490617
Grocery	30.511364	69.488636
Fashion	30.748663	69.251337